

# eLDPC: An Elastic and Scalable LDPC-Decoder With Early Termination by Effectively Leveraging High-Level Synthesis

Shaohua Wang<sup>1</sup>, Zhihao Zeng<sup>1</sup>, Qiang Cao<sup>1</sup>, *Senior Member, IEEE*, Jiawei Yang, Yifan Zhang<sup>2</sup>, Yekang Zhan, and Jie Yao, *Member, IEEE*

**Abstract**—Emerging communication and storage embrace low-density parity-check (LDPC) codes to fully exploit their physical channels. A field-programmable gate array (FPGA) is widely employed to fast prototype and accelerate the LDPC decoding with high complexity. For varying channel conditions, the FPGA decoder is desired to elastically stop iteration when meeting the success condition, avoiding conservatively performing a predefined and large number of iterations. However, the dynamical-execution algorithms with adjustable parameters generally are challenging for a scalable decoder structure, preferred to deterministic execution logic. To overcome the problem, this article presents an elastic and scalable HLS-based FPGA LDPC decoder architecture with early termination to achieve high throughput and flexibility. To this end, eLDPC first provides a universal operation, fully leveraging the features of HLS to efficiently implement optimized small-scale hardware units for low-level data-update operations. Second, eLDPC presents a decoding-iteration pipeline that adds a termination-check (TC) stage to terminate the following iteration for current codeword decoding. eLDPC also presents an HLS-enhanced approach to address memory access conflicts associated with the DU pipeline. Furthermore, eLDPC extends the number of DU decoding-iteration pipelines within a single stream to decode multiple codewords in parallel. Third, eLDPC designs elastic and independent multiple decoding streams by using FIFO queues to decouple the input, output, and a decoding unit (DU) with variable iterations while avoiding the potential blockage of the queueing. We implement and evaluate eLDPC on a Xilinx U55C. Experiments show that eLDPC outperforms recent decoders by up to 5× with the same parameter and achieves the actual decoding throughput of up to 49.5 Gb/s with high scalability and flexibility.

**Index Terms**—Field-programmable gate array (FPGA), high-level synthesis (HLS), low-density parity-check (LDPC).

## I. INTRODUCTION

EMERGING data storage (e.g., NAND flash-based solid-state drives) and communication systems (e.g., 5G and 6G networks) are increasingly exploiting their underlying physical channels to enhance capacity and bandwidth. Low-density parity-check (LDPC) codes, known for their excellent error correction capability close to the Shannon limit, are prevalently used to build high-performance and reliable data channels [1], [2]. Recent researches leverage field-programmable gate array (FPGA) hardware to fast prototype and actually validate their channel designs. Nonetheless, under the high exploitation of these physical channels, the actual signal-to-noise ratio (SNR) is sensitive and varies with its decoding circumstances, e.g., suddenly high in nonideal conditions. Traditional LDPC decoding processes typically employ iterative min-sum message-passing algorithms. In general, the number of iterations as a key threshold is relatively high and fixed to tolerate the worst channel conditions, often at the expense of increased decoding latency. Some decoders adopt early-termination techniques that halt iterations once the LDPC parity-check equations are satisfied. LDPC decoders with early termination techniques have been widely applied in storage and 5G systems [3], [4] to reduce unnecessary decoding iterations; however, these decoders are single-core architectures with limited decoding parallelism, making them difficult to achieve high-throughput [5], [6]. Recently, highly-parallel decoding architectures such as TLP-LDPC [7] and HF-LDPC [8] employ multiple pipelined decoding units (DUs) with static scheduling to increase the overall decoding bandwidth. Nevertheless, these designs without early termination have to perform a large and fixed number of iterations, but avoid significant control complexity and a variable number of decoding iterations introduced by early termination.

To accelerate FPGA development for domain-specific architectures (DSAs), high-level synthesis (HLS) has gained traction as a tool for automatically compiling high-level programming languages like C/C++ into register-transfer level (RTL) implementations. HLS allows developers to focus on algorithmic logic while offloading the intricate and error-prone

Received 12 February 2025; revised 24 June 2025; accepted 16 October 2025. Date of publication 24 October 2025; date of current version 21 May 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFB4505100, in part by NSFC under Grant 62172175, and in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (Shenzhen) under Grant GML-KF-24-24. This article was recommended by Associate Editor Z. Zhang. (*Corresponding author: Qiang Cao.*)

Shaohua Wang, Qiang Cao, Jiawei Yang, Yifan Zhang, and Yekang Zhan are with Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: shaohuawang@hust.edu.cn; caoqiang@hust.edu.cn; wadeyoung@hust.edu.cn; xenon\_zhang@hust.edu.cn; zhanyekang@hust.edu.cn).

Zhihao Zeng is with Guangdong Laboratory of Artificial Intelligence and Digital Economy (Shenzhen), Shenzhen 518107, China (e-mail: mizheng-zzh@foxmail.com).

Jie Yao is with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: jackyao@hust.edu.cn).

Digital Object Identifier 10.1109/TCAD.2025.3625470

tasks of circuit-level layout (e.g., timing and routing control) to the synthesis tool, thereby achieving high productivity and flexibility in hardware configurations and algorithm parameters [9], [10], [11], [12]. However, HLS is still challenging to implement complex logic with the runtime decisions and elastic architectural behaviors upon a large-sized FPGA due to the limited and rigid behavior descriptions of HLS.

This article proposes an elastic HLS-based three-level LDPC-FPGA decoder consisting of scalable DUs with early termination, named eLDPC for short, that effectively leverages HLS to achieve high decoding bandwidth and flexibility in algorithm parameters. In low-level operation modules within DU, eLDPC effectively employs HLS to abstract and optimize primitive message-updating units VN updating unit (VNU), and the CN updating unit (CNU), thus adapting to different LDPC parameters with high hardware efficiency. Second, for mid-level DU, eLDPC presents a pipelined decoding-iteration structure with termination check (TC) and designs an HLS-enhanced approach to address memory access conflicts associated with the DU pipeline. eLDPC further designs multipipelines and a dedicated control within DU to reduce the average latency of decoding a codeword. Third, eLDPC presents an elastic and independent multiple decoding streams. Each stream adopts FIFO queues to decouple the input, output, and DU with variable iterations. eLDPC unleashes the potential of HLS while overcoming HLS's inherent constraints for dynamic algorithms and flexible architectural descriptions.

We implement and evaluate the eLDPC decoder in Xilinx U55C. The results show that eLDPC achieves high scalability, flexibility, and throughput (up to 49.5 Gb/s) and outperforms state-of-the-art (SOTA) implementations with early termination by up to  $5\times$ . Furthermore, eLDPC increases the decoding performance up to 56% under varying channel conditions.

The contributions of this work are as follows.

- 1) We propose an elastic and scalable HLS-based three-level LDPC FPGA decoder architecture to improve the decoding throughput and flexibility under varying channel conditions.
- 2) We abstract low-level CNU and VNU operations to provide implementation flexibility on varying LDPC parameters and hardware bit-widths with high hardware efficiency.
- 3) We present several DU-level techniques, including a DU pipelined decoding iteration with a TC stage, HLS-enhanced approaches to avoid memory access conflicts within the DU pipeline, and a multipipeline structure to decode codewords in parallel.
- 4) We design high-level multiple independent dataflow-based streams using an optimized hardware queue to tolerate the varied execution time of DUs.
- 5) We implement and evaluate the effectiveness of eLDPC in performance, scalability, and flexibility on the Xilinx U55C.

The remainder of this article is organized as follows. Section II introduces the background of LDPC, FPGA, and

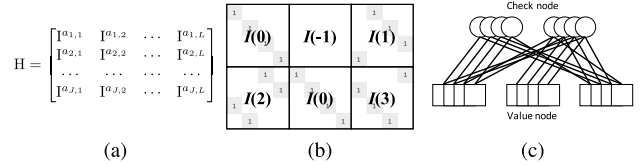


Fig. 1. Example of QC-LDPC PCM and the corresponding Tanner graph. (a) PCM Structure. (b) QC-LDPC PCM. (c) Tanner graph.

HLS. Section III elaborates on the overall design of eLDPC. Section IV presents primitive message updating units (VNU and CNU) within DU. Section V details the design of the DU. Section VI presents multiple decoding streams. Section VII evaluates the performance of eLDPC with SOTA solutions. Section VIII discusses related work. Section IX concludes this article.

## II. BACKGROUND AND MOTIVATION

This section describes the necessary background of LDPC, FPGA, and HLS, and then introduces our motivation.

### A. QC-LDPC

LDPC codes with higher computational complexity generally achieve better error-correction performance. An LDPC code is defined by its parity-check matrix (PCM), where each row and column correspond to a check node (CN) and a variable node (VN), respectively, as shown in Fig. 1(a) and (b). A “1” in the PCM indicates the existence of an edge between the associated VN and CN; all VNs, CNs, and their edges together form a Tanner graph. Fig. 1(c) presents an example of a QC-LDPC code and its corresponding Tanner graph.

The PCM of the example Quasi-Cyclic LDPC (QC-LDPC) code consists of a  $2 \times 3$  array of  $I(n)$  sub-matrices. Each  $I(n)$  is a  $Z \times Z$  circulant permutation matrix obtained by cyclically shifting the identity matrix  $I(0)$  by  $n$  positions, while  $I(-1)$  denotes an all-zero matrix. Since the PCM is constructed by lifting a small base matrix into circulant permutation sub-matrices, hardware implementation complexity can be significantly reduced, as shown in Fig. 2(a). Therefore, a QC-LDPC code and its PCM are fully determined by the base matrix and the expansion factor  $Z$ . This structured form preserves error-correction capability while offering a hardware-friendly architecture.

### B. Decoding Algorithm and Structure

The QC-LDPC decoding typically employs a message-passing-based min-sum algorithm, which converts the complex posterior probability calculations into the logarithmic form, thus reducing computational complexity. Each bit in the codeword is represented by a log-likelihood ratio (LLR) value.

The decoding process iteratively updates LLR messages between CN (CN updating, VNU) and VN (VN updating, CNU) until the stop condition or a predefined iterative upper-threshold is satisfied. Each CN receives messages from its connected VNs according to the check matrix and then

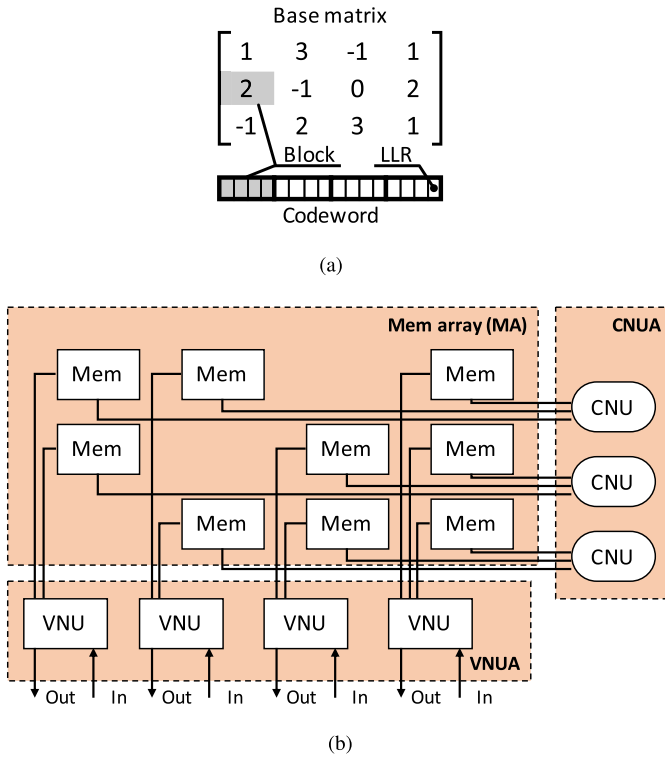


Fig. 2. Interblock parallel structure of a QC-LDPC decoder. (a) Example check matrix and related codeword. (b) QC-LDPC decoder unit.

calculates its new LLRs. When receiving the new messages from its CNs, the VN also updated its LLRs and output the hard decision result.

These sub-matrices of PCM are also referred to as blocks [14]. The initialization values for each column in the block are the corresponding LLR at the corresponding positions “1” in the PCM. Each block is allocated to a memory block, and block-wise circular shifting with a different address offset is used to locate each block, as shown in Fig. 2(b). The memory block stores the received messages after each update.

Computational scheduling of LDPC decoding algorithms is classified into flood and layered update scheduling. The flood scheduling updates all VNs before updating CNs, and vice versa, which has high error-correction capability but low decoding convergence speed. In contrast, the layered scheduling has a faster decoding convergence speed with a limited error-correction capability at a given hardware scale [15]. Besides, the data dependence among layers causes memory-access contention [16]. This work focuses on the flooding scheduling decoding algorithms.

### C. FPGA and HLS

FPGA, as a reconfigurable integrated circuit, has been broadly used for developing domain-specific hardware, from IoT devices to cloud data centers. FPGA is mainly composed of look-up tables (LUT), flip-flops (FF), and routing resources with a specific unit structure and connect topology. Modern FPGAs provide over 200 Gb/s bandwidth of high-bandwidth memory (HBM) comprising multiple memory channels.

FPGA developers traditionally write code using Hardware Description Language (e.g., Verilog) to implement RTL hardware, which is generally tedious and error-prone [17]. Recently, HLS compiled an algorithmic logic behavioral description written in a high-level programming language, such as C/C++, into RTL implementation, achieving high productivity and flexibility [9], [10], [11], [12]. Such HLS-based hardware development has been broadly used in emerging data-intensive applications, e.g., machine learning [18], [19], [20], graph computing [11], [21], and domain-specific accelerators [22], [23].

HLS excels at efficiently optimizing simple computing logic without loops and branches for its RTL implementations. HLS offers pragmas to precisely characterize the behavior-description to improve the quality of HLS results (QoR), such as PIPELINE, UNROLL, and ARRAY\_PARTITION, etc. However, using HLS for complex algorithms still requires a deep understanding of both hardware configuration and the effectiveness of HLS pragmas, including the proper use of pragma directives in the code.

The pragmas also have strict semantics [24], particularly for architectural pragmas such as DATAFLOW and hls::stream. For instance, DATAFLOW allows data to flow efficiently between execution-deterministic modules while supporting the parallel and overlapping execution of the modules. However, HLS is limited by its capabilities in dynamic scheduling when modules' execution times are uncertain. Since the DATAFLOW pragma in HLS does not support the use of control logic, it is necessary to carefully design the control logic required for dynamic execution in each individual module. This limitation significantly constrains HLS's ability to optimize dynamic execution designs. Meanwhile, hls::stream is implemented as a hardware queue with **blocking** mechanism that may lead to entire-dataflow congestion when the queue is empty or full. Therefore, hls::stream is well applicable to scenarios with a continuous and steady data flow.

### D. Motivation

With emerging data storage and communication systems, LDPC codes are widely used to ensure high-performance and reliable data channels under a noise-varying physical channel. However, most LDPC hardware decoders prefer to DUs with a fixed and high number of iterations, which is set to guarantee reliability under the occasionally poor conditions, but sacrificing the throughput at common conditions.

Additionally, modern FPGAs have been increasing their hardware resources. Delivering the growing resources to improve hardware performance. HLS tools offer automation to not only accelerate FPGA design but also to effectively deliver the hardware resources to high performance, but face challenges in handling dynamic execution (e.g., LDPC decoder with early termination). The complex algorithms' logic with runtime decision and large-scale hardware limit HLS's ability to optimize such dynamic designs, resulting in suboptimal performance.

Therefore, we are motivated to design an HLS-based scalable LDPC decoder architecture supporting early termination,

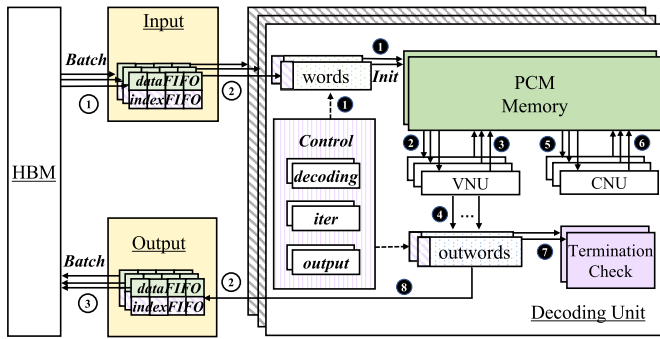


Fig. 3. eLDPC architecture.

thus achieving high decoding throughput while flexibly adapting to varying decoding conditions and parameters.

### III. DECODER DESIGN

We propose an elastic and scalable HLS-based three-level LDPC-FPGA decoder consisting of multiple DUs with early termination, eLDPC for short, as shown in Fig. 3. In the low-level operation module within DU, eLDPC employs HLS to abstract and implement core VNU and CNU (Section IV). This makes eLDPC adapt to different LDPC parameters with high hardware efficiency. Second, for mid-level DU, eLDPC presents a pipelined decoding-iteration structure with a TC. eLDPC further designs multipipelines and a dedicated control within DU (Section V) to enhance the decoding throughput. Third, eLDPC presents a high-level multiple dataflow streams framework (Section VI) to effectively leverage HLS's high-level architectural pragmas while overcoming their inherent behavior constraints.

Specifically, at high-level, eLDPC builds multiple independent decoding-streams, each of which passes three consecutive units as input, DU, and output using *DataFlow*. ① Input reads the corresponding codewords from HBM; ② then DU decodes the codewords with early TC; finally ③ output writes the decoded words back to HBM. These decoding streams operate in parallel and process batches of codewords, thereby significantly improving the data transfer efficiency between HBM and the input/output modules. eLDPC adds hardware queues between input-DU and DU-output for each decoding stream to decouple strict timing requirements among these three units. eLDPC has a similar high-level multi-DUs framework of SOTA HF-LDPC decoder [8], but overcomes variable execution-time of the DU by scheduling the data flow between the input-DU and DU-output. Since each DU performs the full decoding process of its assigned codeword independently, there is no data dependency or interaction among different DUs. Note that in high-level, input, DU, and output also are pipelined, as shown in Fig. 4. The synchronized pipeline in Fig. 4 represents only the temporal alignment of multiple DUs and does not imply any internal parallelization within a single DU.

At mid-level, eLDPC employs interblock parallel flood-scheduling to decode all blocks simultaneously within each DU. Additionally, it designs a fine-grained decoding-iteration pipeline with eight stages using *PIPELINE*

to accelerate computations and reduce the extra latency brought by early TC. The DU can execute a variable number of decoding iterations for codewords. This means that only some codewords perform the predefined maximum iterations under worse conditions. eLDPC further expands the number of DU-pipelines within a stream to decode multiple codewords in parallel.

### IV. VNU AND CNU OPERATIONS

VNU and CNU operations, as the most fundamental computing of LDPC, consist of a series of calculation steps. HLS can automatically analyze dependencies between the calculations at compile time and implement them using small-scale circuits with high QoR.

An LDPC decoding algorithm has multiple parameters, such as bit widths of input and output, each of which has different RTL implementations. To take full advantage of HLS automatic conversion, eLDPC offers an algorithm-parameter abstraction of CNU and VNU functions to concretely perform their corresponding single-round updates within a decoding iteration. These functions are basic computing components in DU.

The specific data type and operation definitions are listed in Tables I and II.

Specifically, the LDPC parameters are classified into three data types: C-bit integer array, single-bit number, and normal integer. Here, C represents the computational precision within the decoder and the input/output precision of the decoder.

Input and output represent the input and output of the CNs and VNs, respectively. They are used to connect to specific memory block arrays based on the connection relationships in the check matrix. The size of the input and output array is the maximum node-degree. Since the functions representing the algorithm will be fully expanded and pipelined at compile time, any unused array member will be removed by HLS, avoiding hardware wastage.

outword is used to output hard decision (1 bit) and the external decoding results during the variable node update. CN\_degree and VN\_degree are used to represent the number of current nodes in the LDPC. Since this variable is a compile-time constant in the design, any integer type can be used.

Algorithm 1 describes CNU and VNU functions. The CNU function first extracts the symbol and magnitude from each input value (line 4), then determines the output symbol by XORing the symbol bits of the other inputs (line 7). Finally, CNU selects the minimum magnitude excluding the current element and combines it with the symbol to produce output (lines 10–13). The VNU function is responsible for updating VNs. It sums all input values to obtain the total sum (line 18), and determines output for hard-decision output based on sum (line 19). Subsequently, for each VN, output[i+1] is computed as sum - input[i+1] with saturation (line 20). Finally, DU's outword retrieves the hard-decision bit from the VNU's output for 1 bit.

The VNU and CNU functions effectively leverage HLS to optimize small-sized serial codes and to implement a fine-grained pipeline. The HLS-function abstraction hides



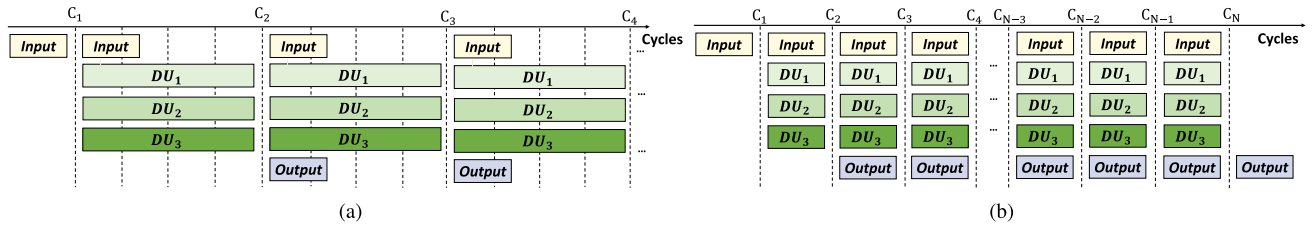


Fig. 4. Decoding pipeline of the high-level decoder. (a) Synchronized pipeline, each DU performs multiple decoding iterations, and all DUs are synchronized according to the longest execution cycle. (b) eLDPC dynamic pipeline, each DU performs one iteration of decoding, and all DUs are aligned in the fine-grain cycle.

TABLE I  
VNU AND CNU FUNCTION DEFINITION

| Definition                             | Description                 |
|----------------------------------------|-----------------------------|
| void VNU(CN_degree, input[], output[]) | Variable node updating unit |
| void CNU(VN_degree, input[], output[]) | Check node updating unit    |

TABLE II  
DATA TYPE OF ALGORITHM FUNCTION

| Parameter | Type  | Data Width | Description                    |
|-----------|-------|------------|--------------------------------|
| input     | array | C bits     | Input of updating unit         |
| output    | array | C bits     | Output of updating unit        |
| outword   | bit   | 1 bit      | Hard decision of updating unit |
| CN_degree | int   | any        | Number of updating unit's port |
| VN_degree | int   | any        | Number of updating unit's port |

implementation differences and the complexity of different parameters for high-level design.

## V. DECODING UNIT

DU, as the core component of the decoder, iteratively performs VNU and CNU until meeting the stopping condition, generally consuming a majority of hardware resources and the longest processing latency. To improve the decoding throughput, first, we propose a decoding-iteration pipeline that fully exploits HLS *PIPELINE*. Second, we design a conflict-free check computation method to address data-access conflicts of the TC stage within the pipeline. Finally, to balance the latency of DU with input and output for the high-level pipeline, we expand the number of parallel decoding-iteration pipelines.

### A. Decoding-Iteration Pipeline

To increase the decoding throughput, eLDPC designs a decoding-iteration pipeline, as shown in Fig. 3. The decoding-iteration comprises the following eight stages in total: ① read data from the FIFO queue to initialize the PCM and words. ② Perform VNU calculation, and ③ write outcomes to the PCM ④ and outwords. ⑤ perform CNU calculation, and ⑥ write outcomes to the PCM. Meanwhile, the TC module reads data from outword. ⑦ Judge whether to terminate the decoding. When successful, ⑧ write the outword to the FIFO queue; otherwise, perform steps ②–⑦ next cycle of high-level again, shown as Fig. 4(b). The FIFO uses HLS *hls::stream* as its hardware queue.

### Algorithm 1 CNU and VNU Functions

```

1: void cnu(int max, cal_data input[],
   cal_data output[])
2: #pragma HLS INLINE
3: for i = 0; i < max; i ++ do
4:   Extract sign & value from input[i]
5: end for
6: for i = 0; i < max; i ++ do
7:   Calculate sign_w[i] by XOR of other signs
8: end for
9: for i = 0; i < max; i ++ do
10:  Find minimum value excluding value_r[i]
11: end for
12: for i = 0; i < max; i ++ do
13:  Combine sign & value to get output[i]
14: end for
15:
16: void vnu(int max, cal_data input[],
   cal_data output[])
17: #pragma HLS INLINE
18: Calculate sum of all input elements
19: Determine output[0] based on sum
20: for i = 0; i < max; i ++ do
21:  Calculate output[i + 1] by sum - input[i + 1] with
   saturation
22: end for

```

The decoding-iteration pipeline uses pragma *PIPELINE* for stages ①–⑧. The pipeline also contains multiple *PCM sub-blocks* (e.g., 64), as shown in Algorithm 2. Stages ⑦ are integrated into the decoding-iteration pipeline, thereby hiding the additional computational latency of the TC.

DU iteratively executes the decoding iteration with eight stages. DU uses an internal counter *iter* to record the count of current iterations for decoding a codeword. When VNU output satisfies the check condition or *iter* reaches its predefined maximal threshold, *iter* is reset to zero. When *iter* is detected to be zero, DU reads a new codeword from the inFIFO and then initializes the PCM.

### B. Data-Access Conflict

VNU decoding is performed in a pipelined manner. However, this pipeline introduces data-access conflicts between the VNU and the TC. Specifically, because the VNU's output can serve as input to the TC module, a data stall may occur if the VNU writes data to the same location in the outward memory block (OMB) that the TC is attempting to read. Concurrent

**Algorithm 2** Decoding Process

```

1: void DU(inFIFO, outFIFO, size,
   max_iteration)
2: if wordnum  $\geq$  size then
3:   return
4: end if
5: if output and  $\neg$ (inFIFO is empty) then
6:   for  $j = 0; j < \text{BLOCK\_SIZE}; j++$  do
7:     #pragma HLS PIPELINE
8:     Initialize word, PCM
9:   end for
10:  output  $\leftarrow$  false; decoding  $\leftarrow$  true
11: end if
12: for  $j = 0; j < \text{BLOCK\_SIZE}; j++$  do
13:   #pragma HLS PIPELINE
14:   Update VNU, outword and PCM
15:   Calculate Termination Check by XOR
16:   Get Termination Check result
17: end for
18: for  $i = 0; i < \text{BLOCK\_SIZE}; i++$  do
19:   #pragma HLS PIPELINE
20:   Update CNU and PCM
21: end for
22: if output and  $\neg$ decoding then
23:   result  $\leftarrow$  0
24: end if
25: if (result = 0  $\vee$  iter = max_iteration) and  $\neg$ output and
    $\neg$ (outFIFO is full) then
26:   Write result to outStream
27:   iter  $\leftarrow$  0; wordnum  $\leftarrow$  wordnum + 1
28:   output  $\leftarrow$  true; decoding  $\leftarrow$  false
29: else if (result = 0  $\vee$  iter = max_iteration) and  $\neg$ output
   and outFIFO is full then
30:   decoding  $\leftarrow$  false
31: end if
32: if  $\neg$ output and decoding then
33:   iter  $\leftarrow$  iter + 1
34: end if

```

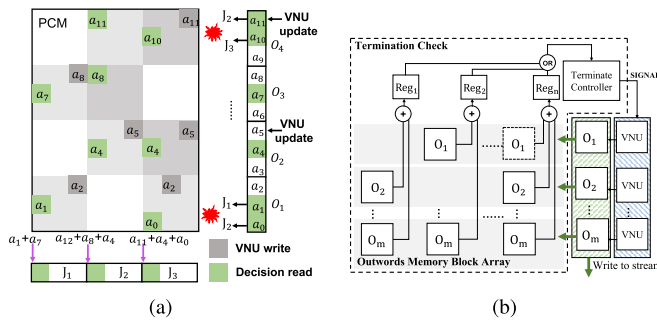


Fig. 5. VNU and TC access memory conflict. (a) Example VNU and TC memory access. (b) Outward memory block array.

access to the OMB also leads to structural hazards when VNU and TC request the memory simultaneously.

For example, as shown in Fig. 5(a), the variable judewords indicates which addresses in the OMB are used for stopping-criteria checks in PCM. Specifically, in the  $O_4$  block, data is read from the addresses  $a_{11}$  and  $a_{10}$ , which are accessed simultaneously by  $J_2$  and  $J_3$ . Similarly, a data hazard arises

when the VNU writes to address  $a_5$  in  $O_2$  at the same time that the TC needs to read from address  $a_4$ .

HLS cannot distinguish between different addresses during compilation. Furthermore, the HLS *PIPELINE* directive parallelizes every loop, leading to conflicts from concurrent memory access. To resolve these conflicts, HLS must stall subsequent loops, which increases the iteration delay.

We address the aforementioned issue by introducing a duplicate OMB. To this end, we introduce an OMB array (OMBA) within the TC, as shown in Fig. 5(b). Subsequently, we then guide HLS to incorporate OMBA ports for enhanced access efficiency, utilizing HLS *ARRAY\_PARTITION* to effectively increase the number of read and write ports for the memory. The decoding results from VNU are written simultaneously into both OMB and OMBA. Consequently, the TC computation can read from different blocks within the same column, avoiding data and structural stalls caused by multiple concurrent read/write operations targeting a single block.

### C. DU Multiwords Parallel Decoding

Note that DU performs an iteration per high-level cycle; each iteration takes eight stages. However, DU requires additional logic for stream management (e.g., stream queue checking, and reading from and writing to the stream, as detailed in Section VI), which slows the DU's decoding process. Since the DU executes one iteration per high-level cycle, its decoding speed can be improved by increasing the number of DU pipelines. Therefore, we increase the number of DU decoding-iteration pipelines to parallel decode multiple codewords per high-level cycle.

1) *In-Order Decoding*: DU must ensure the in-order input and output of a codeword when decoding multiple codewords in parallel because a codeword with fewer iterations could be completed before its ahead-of-codeword.

DU records the order of each codeword in the stream. eLDPC has expanded both the input and output queues. One type of queue is designed for the transmission of codewords (Fig. 3 green part), *dataFIFO*, while the other type records the codewords' input position sequence within the stream (Fig. 3 pink part), *indexFIFO*, thus preserving the correct output order. Each codeword's sequence identification is obtained in the input module. When writing words to the *dataFIFO*, the corresponding sequence ID of the codeword is written to the *indexFIFO* at the same time. DU reads the codeword from the *dataFIFO* and the sequence ID (the pink part in front of *words* and *outwords*) from the *indexFIFO*. During the decoding process, the ID remains unchanged. After the decoding is completed, the sequence ID and the output codeword are written immediately to the *dataFIFO* and *indexFIFO*, and then output to the HBM through the output module for data relocation, as shown in Fig. 3.

With this approach, it can effectively address potential out-of-order issues in parallel decoding, thereby ensuring the correctness of the decoding process.

2) *Control Module*: Due to out-of-order and non-synchronous reading, writing, and handling different codewords to and from the stream, DU requires a control module. The control module not only ensures the correctness of

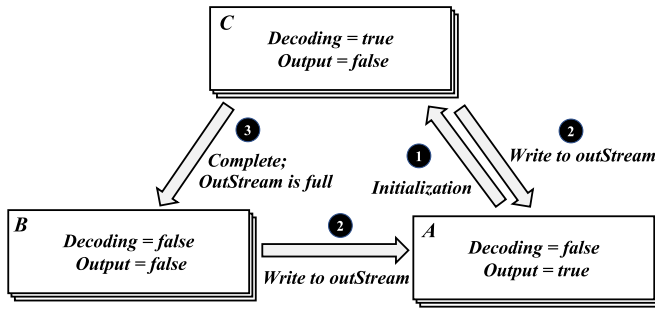


Fig. 6. DU control module state transitions.

decoding codewords but also maintains the stream's queues to avoid being full or empty, making its control module highly complex. Specifically, multiple decoding-iteration pipelines may complete simultaneously and output to the FIFO, leading to a full FIFO queue. In this case, some codewords in the pipelines must wait until the result is output in the next high-level cycle. However, in the next cycle, VNU will update the outward again, which impacts the error correction performance of decoding. Therefore, the control module must handle a variety of complex situations.

Besides, the control module is challenging to integrate efficiently into the DU's decoding pipeline using HLS because it may involve various conditions, leading to numerous conditional branches within loops using *PIPELINE*, making it difficult to achieve an optimal initiation interval (II) of 1.

Therefore, we chose to implement a separate control module to efficiently manage the DU pipelines, as shown in Fig. 6 and Algorithm 2. The control logic maintains the two running states of the DU-pipeline as *output* and *decoding*. When *decoding* is set to true, it indicates that the codeword is being decoded; when *output* is set to true, it indicates that the output to the FIFO queue has been completed. The states *decoding* and *output* can be combined to create three possible states. When *decoding* is false and *output* is true, it indicates that DU has just decoded a codeword, and the result has been written to the FIFO queue. When *decoding* is true and *output* is false, it indicates that the codeword is currently in decoding. When *decoding* is false and *output* is false, it indicates that the codeword has been completely decoded, but output to the FIFO queue has not occurred yet because the FIFO queue is full. Additionally, when *decoding* is false and *output* is false, the codeword will not be updated until its result is written to the FIFO.

By effectively managing the states of *output* and *decoding*, the input and output of codewords can be efficiently controlled.

## VI. MULTISTREAMS

At a high level, to ensure performance scalability with sufficient hardware resources, eLDPC builds multiple independently executed decoding streams in parallel to achieve high throughput. It is very intuitive to use the HLS pragma *DATAFLOW* to describe the decoding stream and to overlap the execution of input, DU, and output modules as a high-level pipeline. The dataflow high-level pipelining is illustrated in Fig. 4(a). *UNROLL* is used to create multiple copies of the decoding stream and makes all streams execute in parallel.

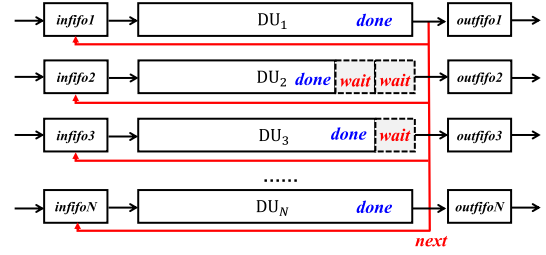


Fig. 7. eLDPC multiple decoding-streams.

### Algorithm 3 eLDPC Stream Processing

```

1: hls::stream infifo[N];
2: hls::stream outfifo[N];
3: for int j = 0; j < size * Iter; j ++ do
4:   #pragma HLS DATAFLOW
5:   input(..., infifo);
6:   for int i = 0; i < N; i ++ do
7:     #pragma HLS UNROLL
8:     DU(infifo[i], outfifo[i]);
9:   end for
10:  output(..., outfifo);
11: end for

```

#### A. DU-Dynamical Execution

The simple combination of *DATAFLOW* and *UNROLL* to build high-level decoding-streams causes all DUs to be strictly synchronized, aligning with the maximal DU latency, which consequently reduces the effectiveness of early-termination. By analyzing the details of the HLS report, within the scope of the *UNROLL* directive, all DUs are aligned with their longest execution latency.

An example is shown in Fig. 7 and its corresponding HLS code in Algorithm 3, which describes the eLDPC multiple decoding-stream framework. Under the outer loop, eLDPC uses *DATAFLOW* to describe the codeword flow between input, DU, and output. Within the inner loop, the pragma *UNROLL* is used to generate multiple independent DUs and implement all DUs that can be executed in parallel.  $D_2$  and  $D_3$  completed the decoding early, but they must wait for a period of time until the other DUs finish decoding before they can read the next codeword from the stream.

HLS cannot resolve this issue because it lacks the necessary abstraction to decouple the dynamic execution timings of individual DUs while maintaining the overall parallelism. Furthermore, HLS also does not support the use of control logic under *DATAFLOW*.

To implement multistreams by leveraging HLS pragmas while decoupling their DUs' synchronization, we first modify the inner *iteration* loop body within the DU to its outer and merge the *iteration* loop with the outermost high-level loop, and an internal runtime variable *iter* is employed to record the count of decoding iterations (details are described in Section V). At a high level, the functionalities of three modules are encapsulated within this new loop body. This means that eLDPC first generates multiple input-DU-output units and can use *UNROLL* to align all parallel units within the finer-grain

cycle. Algorithm 3 shows the eLDPC multistreams code. The loop switch is Line 3.

Note that the execution time of ① and ③ is shorter than the DU processing time, with HLS *DATAFLOW*, ① and ③ can overlap with the execution of DU, thereby hiding the latency of two steps. Fig. 4(a) and (b) show the decoding pipeline of a high-level example of original synchronized and dynamical execution approaches, respectively. The synchronized approach forces all DUs to wait for the slowest DU, which worsens the overall performance. In contrast, the dynamical execution approach decouples all multistreams. In each high-level cycle, DU's execution time is nearly identical and finer than before. Therefore, the dynamic execution approach mitigates the inefficiency caused by synchronous waiting due to iteration variations among DUs.

### B. Stream-DU Decoupling

After decoupling the multiple decoding streams, each codeword goes through steps ①, ②, and ③ independently. Within the DU, codewords are read from the stream only at the start of the DU and are written to the stream at the end. At the beginning, each incoming codeword must initialize the PCM when internal *iter* is zero. At the end of decoding, the results must be output when *iter* reaches the predefined maximum value or decoding stop condition is met (details are provided in Section V). Therefore, codewords do not always flow per cycle, which requires a loosely coupled data flow between the input-DU and DU-output. However, the HLS pragma *DATAFLOW* enforces tightly coupled data transfer between input, DU, and output.

To decouple the DU, input, and output in a data flow stream, we utilize queues to buffer and transfer codewords. Each decoding stream contains an input queue between the input and DU, and an output queue between the DU and output. The commonly utilized HLS datatype *hls::stream*, implemented as a hardware FIFO, is employed to instantiate a queue. In this way, eLDPC implements loosely coupled data flow between the input-DU and DU-output.

### C. Free-Blocking Streams

All data streams in the input module read from the HBM, and all streams in the output module write to the HBM in a synchronous and batched manner, like the HF-LDPC decoder, as shown in Fig. 3 steps ① and ③, to improve data-transfer efficiency.

However, the *hls::stream* datatype has a **blocking** mechanism that may be triggered when trying to write to a full queue or read from an empty queue, potentially causing the entire decoding process to stall, e.g., reading data from HBM memory in a synchronous and batched way and inputting the data into multiple decoding streams. However, due to the dynamic execution of DUs, the number of codewords in each stream may be different, causing some streams to become saturated and then causing the decoding process to block.

To avoid *hls::stream* blockage, eLDPC adds the input and output check for read-empty and write-full conditions for all streams in advance using *hls::stream*'s *full()* and *empty()* to

### Algorithm 4 Output Processing in eLDPC Stream

```

1: void output(..., hls::stream
   outFIFO[batch], int size)
2: static int wordnums = 0;
3: if wordnums ≥ size then
4:   return
5: end if
6: bool EMPTY = 0;
7: for int i = 0; i < batch; i ++ do
8:   #pragma HLS UNROLL
9:   EMPTY = EMPTY | outFIFO[i].empty();
10: end for
11: if EMPTY then
12:   return
13: end if
14: /* outFIFOs write data synchronously to HBM
   memory */

```

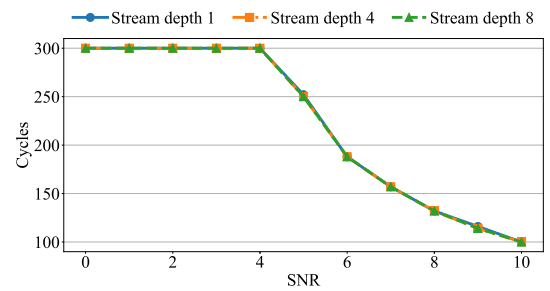


Fig. 8. Execution cycles of stream-depth variations for codewords with different SNRs.

perform an OR operation across all streams. For example, as shown in the Algorithm 4, Lines 6–9, before reading data from *outFIFO*, output first checks the states of all stream queues using an OR operation. If any stream is empty, it skips reading from *outFIFO* and writing to HBM in the current cycle. Input uses a similar method to perform write-full checks.

However, the input/output module checks the stream status at the beginning of execution and skips later operations in the current cycle, which may increase total execution cycles and decrease the overall performance. We conducted a simulation experiment to analyze this issue.

The simulation experiment evaluates the decoder's performance with ten decoding streams by measuring the number of executions of the high-level loop, representing the total time in hardware. Additionally, each experiment uses the same dataset, consisting of 300 codeword groups for each SNR condition. Therefore, each decoding stream decodes 30 codewords. The predefined maximum number of iterations is fixed at 10, meaning that the maximum number of execution cycles is 300.

We change the largest number of codewords accommodated in the streams (i.e., the stream depth) to evaluate the impact of the input/output checks on decoding performance. We used streams with depths of 1, 4, and 8, respectively. As shown in Fig. 8, the total execution cycles across the three different stream depths are remarkably consistent. The stream with a depth of 8 theoretically experiences far fewer blockages compared to the stream with a depth of 1, yet their final total execution cycles remain the same. This empirical observation



suggests that the input/output checks and skip later operations are unrelated to the queue depth.

## VII. EXPERIMENT AND EVALUATION

### A. Implementation and Experimental Setup

We use the Xilinx Vitis hardware development platform to implement a set of eLDPC decoders on an FPGA and conduct experiments to analyze the performance and hardware consumption. The experimental platform employs a server with an AMD Ryzen 7-7700X and 62 GB of memory. The FPGA used for the experiment is a Xilinx Alveo U55C data center accelerator card with the official firmware (xilinx\_u55c\_gen3  $\times$  16\_xdma\_3\_202210\_1). The maximum clock frequency of the FPGA is 500 MHz, and the data width of HBM is 512 bits. U55C supports up to 32 independent HBM channels, enabling high-bandwidth parallel data access and significantly improving large-scale data throughput.

The decoder is configured to follow the 802.16e standard, supporting various code rates ranging from 0.5 to 0.9. Each codeword consists of 1536 symbols defined over the Galois Field GF(2), with each symbol represented by a LLR scalar during decoding. The PCM block size is set to 64, so each codeword and PCM are divided into 24 blocks. The input module reads a codeword of 24 LLRs from each stream in a single HBM channel at a time, which requires 64 read operations to fully retrieve the codeword. Additionally, eLDPC supports parallel computation with multiple decoding streams, and the number of decoding streams varies based on the codeword width. For example, when the LLR width is 2 bits, the total LLR width read from a single HBM channel per block is 48 ( $24 \times 2$ ) bits for each stream. To avoid exceeding the maximum width of an HBM channel, the decoder can support up to ten parallel decoding streams for reading data. Furthermore, the number of DUs in eLDPC can be further scaled based on the number of available HBM channels, thereby enhancing parallel processing capability and overall system throughput. In the experiments, the test data were generated through simulations using standard LDPC coding and BPSK-AWGN channel simulation techniques [28]. Multiple information sequences, consisting of 0 and 1 s, were simulated over a range of SNR from 0 to 10 to generate the test data. For each SNR, a total of 3000 decoded codewords were simulated. In the experimental setup, the maximum preset number of iterations for the decoder was 15. Finally, these codewords were combined in various ways to test the performance of eLDPC.

By default, in a fully featured decoder, an eLDPC DU uses four DU-pipelines, called, implemented with an eLDPC\_4 Section VII-C. In addition, LDPC-FT, a baseline decoder based on the implementation in R-LDPC [29] with a fixed iteration number; an HLS decoder with early TC (eLDPC-ET), an elastic and scalable decoder with multiple decoding streams (eLDPC\_N). In eLDPC and eLDPC\_4, the queues in the streams can buffer a maximum of 8 decoded codewords.

### B. Overall Performance

Table III lists the decoding performances of this work and SOTA decoders; iterations marked with \* indicate the results

with early termination. Furthermore, for a fair comparison, we implemented multiple versions of eLDPC\_4 with the same hardware configuration and LDPC parameters as the existing decoders.

Table III shows that the decoding throughput of eLDPC\_4 achieves up to 49.5 Gb/s. We also test the performance of HF-LDPC with a fixed number of 15 iterations under the same parameters. HF-LDPC employs an interleaved updating task-level pipeline within a DU to execute VNU and CNU in parallel, and achieved an overall throughput of 45.6 Gb/s. eLDPC\_4 outperforms the other implementations that use early termination by up to  $5\times$  under the same parameters.

Besides, Delomier et al. [26] employ horizontal layered scheduling and enable parallel decoding across multiple frames, but confine each parallelized unit to operate on subdivided blocks within the same frame. This architectural constraint potentially limits the scalability of DUs at the hardware level. In comparison, eLDPC 4 adopts a multistream and multi-DU architecture, where each DU independently processes one or more complete codewords. The number of DUs can be linearly scaled to over 80, enabling efficient utilization of available FPGA resource pools and facilitating large-scale parallel decoding.

The eLDPC adopts a multistream structure that can scale to utilize more hardware resources. eLDPC consumes relatively more LUT and FF resources; however, its efficiency (decoding throughput divided by hardware consumption) surpasses both existing HLS-based and even RTL-based solutions. eLDPC significantly reduces the usage of BRAM and URAM; this is because HLS maps only top-level function variables to BRAM by default, and URAM must be explicitly declared. Therefore, only hardware queues within each decoding stream utilize a small amount of BRAM, leading to a more efficient overall memory architecture. In summary, eLDPC\_4 fully leverages HLS capabilities and hardware resources to achieve high throughput, flexibility, and scalability.

### C. DU Structure and Number of DU-Pipelines

We conducted an empirical evaluation of the decoding performance of different DU structures to understand their effectiveness. We implement three different designs: DU-FT, DU-ET, and DU-ET-N under varying SNRs. DU-FT is a DU pipeline without ET. DU-ET extends the DU-FT pipeline by adding the TC. DU-ET-N, which includes the TC, modifies DU-ET to perform N pipelines in parallel. We measure their decoding throughputs, as shown in Fig. 9(a). DU-FT exhibits a constant decoding throughput of about 1 Gb/s under an SNR range because it performs fixed decoding iterations. In contrast, DU-ET and DU-ET-1 increase their decoding throughputs as the SNR increases. However, the performance of DU-ET-1 is approximately half that of DU-ET. DU-ET-1 only executes one iteration but introduces extra operations (e.g., stream queue state check, read from, and write to stream) per stream-level cycle, thus degrading the performance. However, the DU-ET-N structure can add the number of DU-pipelines.

We conducted a simulation experiment to verify the effectiveness of increasing the number of DU-pipelines (DU-ET-N).

TABLE III  
DECODER PERFORMANCE COMPARISON

| Design  | Technology | Length | Rate | Width | Max iter. | Latency(ms) | Thpt.(Gbps) | kLUTs  | kFFs  | BRAM | Efficiency. | Freq.  |
|---------|------------|--------|------|-------|-----------|-------------|-------------|--------|-------|------|-------------|--------|
| eLDPC_4 | HLS        | 1536   | 1/2  | 2     | 15*       | 0.621       | 49.5        | 500    | 270.2 | 584  | 101.74      | 300MHz |
| eLDPC_4 | HLS        | 1536   | 1/2  | 3     | 15*       | 0.792       | 35.10       | 315.8  | 191.3 | 132  | 81.38       | 300MHz |
| eLDPC_4 | HLS        | 1536   | 1/2  | 4     | 15*       | 1.049       | 30.07       | 420.07 | 256.5 | 308  | 73.30       | 300MHz |
| eLDPC_4 | HLS        | 1536   | 1/2  | 6     | 15*       | 1.304       | 22.23       | 348.7  | 205.4 | 90   | 65.28       | 300MHz |
| eLDPC_4 | HLS        | 1536   | 3/4  | 6     | 15*       | 1.162       | 27.34       | 350.08 | 199.3 | 365  | 69.97       | 300MHz |
| [8]     | HLS        | 1536   | 1/2  | 2     | 15        | 0.652       | 45.6        | 382.82 | 327.3 | 180  | 128.65      | 300MHz |
| [7]     | HLS        | 1536   | 1/2  | 6     | 10        | 2.825       | 9.63        | 385.5  | -     | -    | 25.58       | -      |
| [25]    | RTL        | 2304   | 1/2  | 4     | 10*       | -           | 0.34        | 12.3   | 0.8   | 24   | 27.84       | 150MHz |
| [26]    | HLS        | 1152   | 1/2  | -     | 10        | 2.567       | 0.317       | 10.9   | 21.66 | 32   | 28.1        | 300MHz |
| [27]    | RTL        | 1296   | 1/2  | 3     | 20        | -           | 3.36        | 80.3   | -     | 104  | 42.84       | 233MHz |

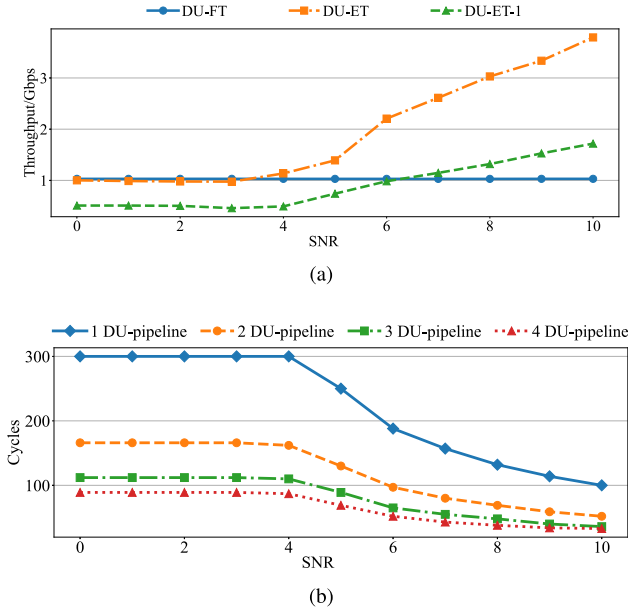


Fig. 9. Performance of the decoder with different DU structures. (a) Actual decoding throughput at different SNRs. (b) High-level cycles of different DU-pipelines at different SNRs.

The experimental setup maintains identical configuration parameters to those described in Section VI-C. We modified the number of DU decoding pipelines within a stream to decode multiple codewords in parallel. As shown in Fig. 9(b), with the increasing number of DU's decoding-pipelines, the overall execution cycles of the decoder continuously decrease.

However, the increasing number of DU pipelines also consumes hardware resources significantly. We found that the DU-ET-4 is a tradeoff between the overall throughput and hardware efficiency because codewords with an SNR in the 4–10 range almost need four decoding iterations on average.

#### D. SNR

We conducted two sets of experiments to evaluate the performance of the decoders in different SNR conditions. In comparison to eLDPC, eLDPC-ET, and the LDPC-FT, eLDPC\_4 demonstrated superior performance across both experimental scenarios.

1) *Different SNR Conditions:* We first compare the decoding performance of data generated at different SNRs.

As depicted in Fig. 10(a), both eLDPC\_4, eLDPC, and eLDPC-ET all demonstrate an increasing throughput as the SNR rises, whereas the throughput of LDPC-FT remains constant. This is due to the improved signal quality of the channel with higher SNR, allowing eLDPC and eLDPC-ET to take advantage of early termination on low-error codewords. In contrast, the LDPC-FT uses a fixed number of iterations, resulting in a fixed iteration number regardless of SNR changes. Once the SNR surpasses three, the throughput of eLDPC-ET and eLDPC gradually increases as early termination becomes more frequent. Compared to eLDPC-ET, eLDPC\_4 demonstrates significant advantages compared to eLDPC-ET under low SNR conditions, achieving higher decoding throughput while maintaining fewer fluctuations. This indicates that eLDPC\_4 is capable of adapting to a wide range of SNR conditions with stable performance. As the SNR increases, the performance of eLDPC\_4 approaches to that of eLDPC-ET, primarily because the additional logic operations in eLDPC\_4 to ensure decoder reliability introduce some computational overhead, reducing the performance advantage in high SNR conditions.

2) *Burst SNR Variations:* In this section, we arrange the codewords with known decoding iterations by interleaving them based on their iteration differences to form the test dataset. We assess the adaptability of eLDPC, eLDPC\_4, to burst changes in signal quality by interleaving codewords from different SNR channels (1, 2, 3, 4, 5, 7, and 10) for the decoding process. As shown in Fig. 10(b), eLDPC\_4 outperforms the other decoders in this situation. When the SNR difference among codewords is less than 7, the decrease in throughput for eLDPC and eLDPC\_4 is minimal compared to LDPC-FT and eLDPC-ET. This is because the *stream* feature in eLDPC can buffer up to 8 codewords, mitigating the variation of SNR across different streams when the burst SNR variation reaches 7 or higher, both eLDPC and eLDPC\_4 experience significant performance degradation.

3) *Variation in Maximum Fixed Iterations:* In this section, we compare the performance of different decoders with varying maximum fixed iteration numbers across four SNRs, as depicted in Fig. 10(c)–(f). All decoders achieve high throughputs and complete decoding within a relatively small number of iterations (less than five).

However, when the SNR is 0, these four decoders exhibit a similar behavior with a significant reduction in throughput as

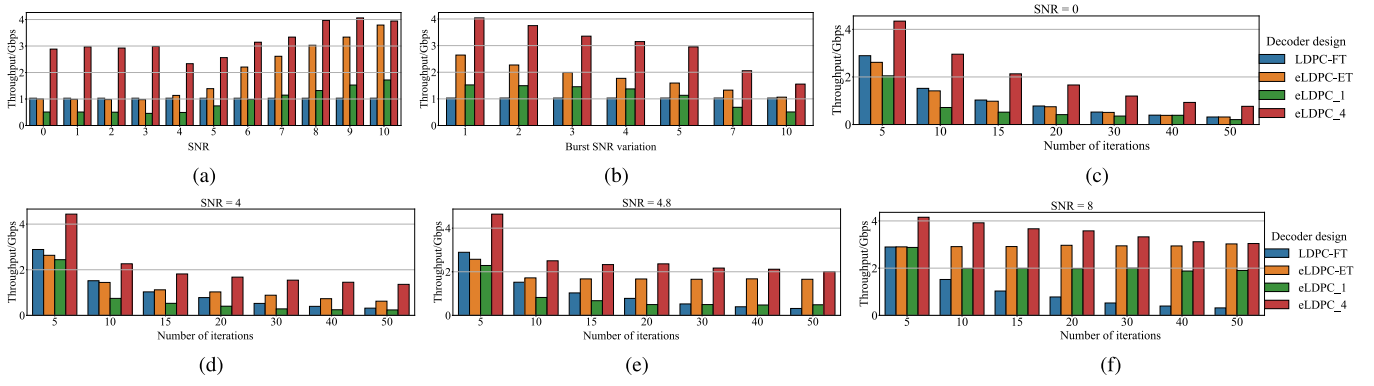


Fig. 10. Decoder throughput of eLDPC in different scenarios. (a) Actual decoding throughput at different SNRs. (b) Actual decoding throughput for codewords with burst SNR variations. (c) Actual decoding throughput for codewords at an SNR of 0 with varying fixed maximum iterations. (d) Actual decoding throughput for codewords at an SNR of 4 with varying fixed maximum iterations. (e) Actual decoding throughput for codewords at an SNR of 4.8 with varying fixed maximum iterations. (f) Actual decoding throughput for codewords at an SNR of 8 with varying fixed maximum iterations.

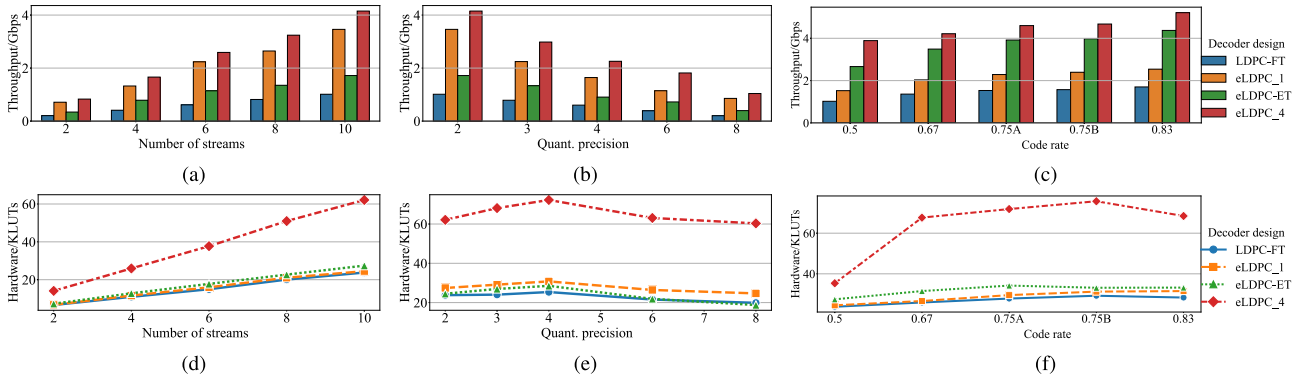


Fig. 11. Scalability and flexibility of the eLDPC decoder across varying numbers of streams, code rates, and quantization precision. (a) Actual decoding throughput at different numbers of streams. (b) Actual decoding throughput for codewords with varying quantification precision. (c) Actual decoding throughput for codewords with varying code rates. (d) Actual hardware resource at different numbers of streams. (e) Actual hardware resource for codewords with varying quantization precision. (f) Actual hardware resource for codewords with varying code rates.

the iteration number increases. This is because all decoders have difficulty meeting the stop conditions when the SNR is 0, although eLDPC-ET, eLDPC, and eLDPC\_4 have the capability to early terminate decoding, but such occurrences are rare.

When the SNR exceeds 4.8, the throughputs of eLDPC, eLDPC\_4, and eLDPC-ET become similar as the number of iterations increases. Their throughputs remain relatively stable due to the early termination with the tenth iteration, while the LDPC-FT while LDPC-FT experiences a significant throughput decrease. Overall, eLDPC\_4 consistently outperforms both eLDPC and eLDPC-ET, maintaining a higher throughput.

### E. Scalability and Flexibility

1) *Scalability*: To further understand the scalability of eLDPC\_4, we adjust the number of streams to test the decoding throughput of the decoder.

Fig. 11 shows that the eLDPC decoder scales linearly with the increasing number of streams. It also suggests that HLS is convenient and effective for scaling its performance under large-scale hardware.

2) *Flexibility*: In addition to scalability, we test the flexibility of the eLDPC design by adopting different LDPC code

rates and adjusting the bit-width of the LDPC decoder's calculations. This is achieved by simply replacing the key arrays and constants in the HLS code for eLDPC behavior description, making it adaptable to varying hardware configurations.

The code rate, which is the ratio of effective information bits to the total transmitted bits during both encoding and decoding, plays a crucial role in determining decoding efficiency. In this work, we chose two different 0.75 LDPC code rates. Fig. 11 shows that eLDPC\_4 can effectively adapt to different code rates and data quantization precisions. The decoder shows substantial flexibility, maintaining high performance across different configurations. In all tested scenarios, eLDPC\_4 improves hardware efficiency by as much as 4× compared to the LDPC-FT design.

3) *5G*: Furthermore, we implemented the eLDPC decoder based on the 5G standard, targeting LDPC codewords with a code rate of 0.5 and a length of 4352. Under this configuration, each HBM channel can support at most three parallel DUs. In this experiment, we activated only a single HBM channel to quickly evaluate the architectural flexibility of eLDPC. As shown in Table IV, with three DUs running in parallel, eLDPC still achieves a decoding throughput of 1 Gb/s, demonstrating its high performance and flexibility under long-codeword configurations.

TABLE IV  
DECODER PERFORMANCE OF 5G STANDARD

| DUs | Throughput (Gbps) | kLUTs | kFFs | BRAM |
|-----|-------------------|-------|------|------|
| 1   | 0.4               | 13.5  | 0.7  | 7    |
| 2   | 0.78              | 14.1  | 10.3 | 11   |
| 3   | 1.0               | 20.2  | 13.8 | 14   |

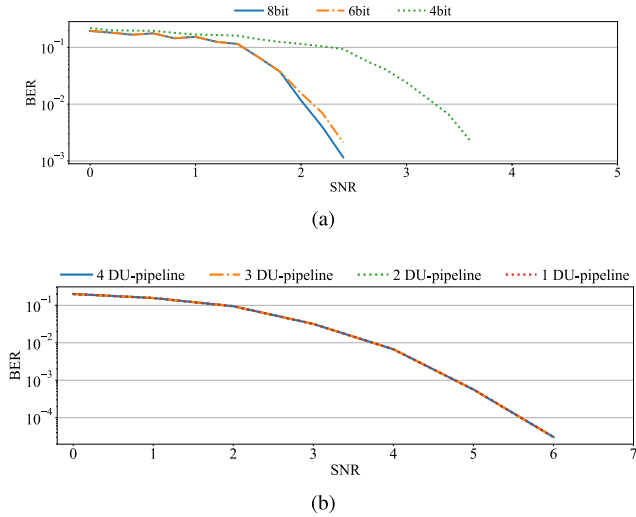


Fig. 12. Error correction performance. (a) Impact of quantization bit-width on BER performance. (b) Impact of DU-pipeline parallelism on BER performance. The results are obtained for the 802.16e standard LDPC code with a code rate of 0.5, code length of 1536, and 15 decoding iterations.

#### F. Error Correction Performance

To further evaluate the error correction performance of eLDPC\_4, we evaluate the bit error rate (BER) under various SNR conditions for different quantization precisions.

Fig. 12 shows the BER for eLDPC\_4 with different quantization bit-widths (4-bit, 6-bit, and 8-bit) for varying SNR values. The results demonstrate that eLDPC\_4 maintains an excellent error correction performance even at lower quantization precision. As shown in Fig. 12, higher bit-width quantization levels result in a lower BER, with 8-bit quantization achieving the best results. It is worth noting that under 4-bit, 6-bit, and 8-bit quantization configurations, each HBM channel can accommodate up to 5, 3, and 2 parallel DUs, respectively. This indicates that eLDPC\_4 can maintain reliable decoding accuracy under varying numbers of parallel DUs. In conclusion, eLDPC\_4 maintains high decoding throughput while delivering excellent error correction performance.

In addition, we further analyzed the impact of varying DU-pipeline numbers on the error correction performance of eLDPC. We evaluated decoders configured with different numbers of pipelines using 2-bit bit-widths. The results show that the BER curves converge at an SNR of 6, primarily due to the slower convergence behavior associated with low-precision inputs. Notably, the number of DU-pipelines has minimal effect on decoding accuracy, which can be attributed to the index queue mechanism in the eLDPC that ensures the consistency and ordering of the output data.

### VIII. RELATED WORK

#### A. HLS-Based LDPC Decoder

Wang et al. [30] implement a performance-balanced, general-purpose QC-LDPC decoder design using Vivado HLS, which is similar to the RTL design in 2007 [14], [31]. However, this work only adopts interblock parallelism and does not eliminate the dependency between VNU and CNU in the flooding decoding algorithm. As a result, the efficiency of the design is relatively low.

Zhang et al. [8] proposed an HLS-friendly high-throughput LDPC decoder design that can flexibly support the implementation for various QC-LDPC codes. However, this work did not abstract the algorithm nor design indirect constant propagation. Thus, it cannot flexibly and efficiently support the implementation of different LDPC decoding algorithms.

Mhaske et al. [32], [33] propose a layered decoder based on LabVIEW-based HLS. This HLS approach represents the hardware design using circuit diagrams, which have a larger semantic gap than mainstream HLS based on software programming languages. Additionally, due to the insufficient exploitation of interblock and intrablock parallelism, the throughput per unit of hardware is generally lower.

Zhang [8] presents an HLS-based LDPC decoder with a two-level pipelined architecture. However, they perform fixed decoding iterations and depend on deterministic execution cycles of each unit and strict data balance between interunits to achieve high throughput. These subtle and highly parallel architectures employ a fixed, even long iteration count according to the tolerated worst case of the physical channel to avoid the complexity and uncertainty of adaptive decoding termination.

Delomier et al. [26] propose a model-based LDPC decoder design using SystemC and Vivado HLS, featuring two-layered Min-Sum decoder models (M1 and M2) with SIMD-style intraframe parallelism. The design achieves high hardware efficiency. However, it uses fixed decoding iterations without early termination, leading to unnecessary computation under good channel conditions and limiting adaptivity in latency and power reduction.

eLDPC partially draws on architectural designs of these HLS-based LDPC decoders but introduces early termination of decoding with high logic complexity.

#### B. LDPC Decoder With Early Termination

Existing early-termination approaches are classified into algorithm and hardware decision methods. The formers [34], [35] design termination criteria in their algorithm logic, thus avoiding on-the-fly hardware decisions, but are severely limited to the dedicated algorithms without generality.

On the other hand, the latter [36], [37] introduce a specific early TC unit, allowing variable iteration counts based on stop conditions. Park et al. [38] propose a node-level convergence detection method that eliminates the need for syndrome value calculations. However, this approach requires a minimum of several iterations to detect convergence, leading to a higher number of iterations in the high SNR region compared to the standard ET scheme.



Li et al. [36] propose an improved Trellis Min-Max (T-MM) algorithm and its associated architecture. T-MM is based on the characteristics of LLR message computation and uses specific layer conditions to determine whether the decoding has converged without the need to compute parity checks. This approach reduces the average number of iterations without sacrificing error rate performance.

Balatsoukas-Stimming and Dollas [37] propose a very high speed LDPC code decoder architecture for (3, 6)-regular codes by employing hybrid quantization, pipelining, and FPGA-specific optimizations, with an early termination circuit is employed. Their pipelined architecture fully addresses the decoder's significant I/O requirements.

These hardware decoders are implemented as single-layer DUs without a scalable parallel decoding architecture and code flexibility, which are goals eLDPC achieves in this article.

## IX. CONCLUSION

We design an elastic LDPC FPGA decoder architecture eLDPC. eLDPC first employs a decoding iteration pipeline that adds a TC stage to terminate decoding. eLDPC further presents multiple decoding streams, each consists of a DU with variable iterations to decode independently. eLDPC outperforms SOTA implementations with early termination by up to  $5\times$  and achieves up to 49.5 Gp/s. The experiments also validate that the eLDPC has effectiveness, scalability, and flexibility.

## REFERENCES

- [1] E. Yeo, B. Nikolic, and V. Anantharam, "Architectures and implementations of low-density parity check decoding algorithms," in *Proc. 45th Midwest Symp. Circuits Syst. (MWSCAS)*, vol. 3, Aug. 2002, pp. III-437–III-440.
- [2] H. Park, D. Lee, and J. Moon, "LDPC code design for distributed storage: Balancing repair bandwidth, reliability, and storage overhead," *IEEE Trans. Commun.*, vol. 66, no. 2, pp. 507–520, Feb. 2018.
- [3] J. Jung, "Energy-efficient partial LDPC decoding for NAND flash-based storage systems," *Electronics*, vol. 13, no. 7, p. 1392, Apr. 2024.
- [4] N.-I. KIM and J.-U. KIM, "Early termination scheme for 5G NR LDPC code," in *Proc. Int. Conf. Inf. Commun. Technol. Conver. (ICTC)*, Oct. 2021, pp. 933–935.
- [5] C.-H. Lin, Y.-S. Wu, and C.-P. Song, "Energy-efficient LDPC codec design using cost-effective early termination scheme," *IET Comput. Digit. Techn.*, vol. 13, no. 2, pp. 118–125, Mar. 2019.
- [6] J. A. Kennedy and D. L. Noneaker, "Scheduling parity checks for increased throughput in early-termination, layered decoding of QC-LDPC codes on a stream processor," *EURASIP J. Wireless Commun. Netw.*, vol. 2012, no. 1, pp. 1–10, Dec. 2012.
- [7] Y.-F. Zhang, L. Sun, and Q. Cao, "TLP-LDPC: Three-level parallel FPGA architecture for fast prototyping of LDPC decoder using high-level synthesis," *J. Comput. Sci. Technol.*, vol. 37, no. 6, pp. 1290–1306, Dec. 2022.
- [8] Y. Zhang, Q. Cao, S. Wang, J. Yao, and H. Jiang, "HF-LDPC: HLS-friendly QC-LDPC FPGA decoder with high throughput and flexibility," in *Proc. IEEE 41st Int. Conf. Comput. Design (ICCD)*, Nov. 2023, pp. 566–573.
- [9] J. Wang, L. Guo, and J. Cong, "AutoSA: A polyhedral compiler for high-performance systolic arrays on FPGA," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2021, pp. 93–104.
- [10] L. Lu et al., "TENET: A framework for modeling tensor dataflow based on relation-centric notation," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2021, pp. 720–733.
- [11] X. Chen, H. Tan, Y. Chen, B. He, W.-F. Wong, and D. Chen, "ThunderGP: HLS-based graph processing framework on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, New York, NY, USA: ACM, Feb. 2021, pp. 69–80.
- [12] P. G. Mousoulitis and L. P. Petrou, "CNN-grinder: From algorithmic to high-level synthesis descriptions of CNNs for low-end-low-cost FPGA SoCs," *Microprocessors Microsyst.*, vol. 73, Mar. 2020, Art. no. 102990.
- [13] R. Townsend and E. Weldon, "Self-orthogonal quasi-cyclic codes," *IEEE Trans. Inf. Theory*, vol. IT-13, no. 2, pp. 183–195, Apr. 1967.
- [14] Z. Wang and Z. Cui, "A memory efficient partially parallel decoder architecture for quasi-cyclic LDPC codes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 15, no. 4, pp. 483–488, Apr. 2007.
- [15] M. Milicevic and P. G. Gulak, "A multi-Gb/s frame-interleaved LDPC decoder with path-unrolled message passing in 28-nm CMOS," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 10, pp. 1908–1921, Oct. 2018.
- [16] K. Zhang, X. Huang, and Z. Wang, "High-throughput layered decoder implementation for quasi-cyclic LDPC codes," *IEEE J. Sel. Areas Commun.*, vol. 27, no. 6, pp. 985–994, Aug. 2009.
- [17] R. Nane et al., "A survey and evaluation of FPGA high-level synthesis tools," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1591–1604, Oct. 2016.
- [18] A. Sohrabzadeh, C. H. Yu, M. Gao, and J. Cong, "AutoDSE: Enabling software programmers design efficient FPGA accelerators," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, New York, NY, USA: ACM, Feb. 2021, p. 147.
- [19] P. Goswami, M. Shahshahani, and D. Bhatia, "MLSBench: A synthesizable dataset of HLS designs to support ML based design flows," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, New York, NY, USA: ACM, Feb. 2020, p. 312.
- [20] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2015, pp. 161–170.
- [21] G. Falcao, V. Silva, L. Sousa, and J. Andrade, "Portable LDPC decoding on multicores using OpenCL [applications corner]," *IEEE Signal Process. Mag.*, vol. 29, no. 4, pp. 81–109, Jul. 2012.
- [22] J. Li, Y. Chi, and J. Cong, "HeteroHalide: From image processing DSL to efficient FPGA acceleration," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, Feb. 2020, pp. 51–57.
- [23] A. Mehrabi, A. Manocha, B. C. Lee, and D. J. Sorin, "Prospector: Synthesizing efficient accelerators via statistical learning," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2020, pp. 151–156.
- [24] Xilinx. *Vitis Data Compression Library*. Accessed: Jan. 5, 2025. [Online]. Available: <https://github.com/Xilinx/VitisLibraries/tree/main/datacompression>
- [25] S. Yesil and M. Arslan, "Dual port ram based layered decoding for multi rate quasi-cyclic LDPC codes," in *Proc. 12th Int. Conf. Signal Process. (ICSP)*, Oct. 2014, pp. 1524–1530.
- [26] Y. Delomier, B. Le Gal, J. Crenne, and C. Jego, "Model-based design of flexible and efficient LDPC decoders on FPGA devices," *J. Signal Process. Syst.*, vol. 92, no. 7, pp. 727–745, Jul. 2020.
- [27] S. Nimara, O. Boncalo, A. Amaricai, and M. Popa, "FPGA architecture of multi-codeword LDPC decoder with efficient BRAM utilization," in *Proc. IEEE 19th Int. Symp. Design Diag. Electron. Circuits Syst. (DDECS)*, Apr. 2016, pp. 1–4.
- [28] P. Hailes, L. Xu, R. G. Maunder, B. M. Al-Hashimi, and L. Hanzo, "Survey of FPGA-based LDPC decoders," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1098–1122, 2nd Quart., 2016.
- [29] Y. Zhang, Q. Cao, J. Yao, and H. Jiang, "R-LDPC: Refining behavior descriptions in HLS to implement high-throughput LDPC decoder," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Apr. 2023, pp. 1–6.
- [30] B. Wang, J. Kang, and Y. Zhu, "Performance balanced general decoder design for QC-LDPC codes using Vivado HLS," in *Proc. IEEE 11th Int. Conf. Electron. Inf. Emergency Commun. (ICEIEC) IEEE 11th Int. Conf. Electron. Inf. Emergency Commun. (ICEIEC)*, Jun. 2021, pp. 26–30.
- [31] J. Andrade, F. Pratas, G. Falcao, V. Silva, and L. Sousa, "Combining flexibility with low power: Dataflow and wide-pipeline LDPC decoding engines in the Gbit/s era," in *Proc. IEEE 25th Int. Conf. Appl.-Specific Syst., Archit. Processors*, Jun. 2014, pp. 264–269.
- [32] S. Mhaske, H. Kee, T. Ly, A. Aziz, and P. Spasojevic, "FPGA-based channel coding architectures for 5G wireless using high-level synthesis," *Int. J. Reconfigurable Comput.*, vol. 2017, no. 1, pp. 1–23, 2017.
- [33] S. Mhaske, D. Uliana, H. Kee, T. Ly, A. Aziz, and P. Spasojevic, "A 2.48Gb/s FPGA-based QC-LDPC decoder: An algorithmic compiler implementation," in *Proc. 36th IEEE Sarnoff Symp.*, Sep. 2015, pp. 88–93.

- [34] A. Hera et al., "Analysis and implementation of on-the-fly stopping criteria for layered QC LDPC decoders," in *Proc. 22nd Int. Conf. Mixed Design Integr. Circuits Syst. (MIXDES)*, Jun. 2015, pp. 287–291.
- [35] M. Ferrari, S. Bellini, and A. Tomasoni, "Safe early stopping for layered LDPC decoding," *IEEE Commun. Lett.*, vol. 19, no. 3, pp. 315–318, Mar. 2015.
- [36] M.-R. Li, W.-X. Chu, H.-C. Lee, and Y.-L. Ueng, "An efficient high-rate non-binary LDPC decoder architecture with early termination," *IEEE Access*, vol. 7, pp. 20302–20315, 2019.
- [37] A. Balatsoukas-Stimming and A. Dolas, "FPGA-based design and implementation of a multi-GBPS LDPC decoder," in *Proc. 22nd Int. Conf. Field Program. Log. Appl. (FPL)*, Aug. 2012, pp. 262–269.
- [38] Y. S. Park, Y. Tao, and Z. Zhang, "A fully parallel nonbinary LDPC decoder with fine-grained dynamic clock gating," *IEEE J. Solid-State Circuits*, vol. 50, no. 2, pp. 464–475, Feb. 2015.



**Shaohua Wang** received the B.S. degree in computer science from Huazhong Agricultural University, Wuhan, China, in 2021. He is currently pursuing the M.S. degree in computer architecture with Wuhan National Laboratory for Optoelectronics, Wuhan.

His research interests mainly include field programmable gate arrays and error correcting code and storage systems.



**Zhihao Zeng** received the Ph.D. degree in optical engineering from Huazhong University of Science and Technology, Wuhan, China, in 2024.

He is currently an Associate Researcher with Guangdong Laboratory of Artificial Intelligence and Digital Economy (Shenzhen), Shenzhen, China. His research interests include several areas of applied physics and emerging technologies, especially in the research on next-generation data storage mechanisms and methods.



**Qiang Cao** (Senior Member, IEEE) is currently a Full Professor with Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan, China. His research interests include computer architecture, large-scale storage systems, and performance evaluation.

Prof. Cao is a member of ACM. He is a distinguished member of China Computer Federation (CCF).



**Jiawei Yang** received the B.S. degree in biological science from Wuhan University, Wuhan, China, in 2022, and the M.S. degree in computer science from Huazhong University of Science and Technology, Wuhan, in 2025.

His research focuses on FPGA-based hardware acceleration, including high-throughput QC-LDPC encoders/decoders design using HLS and heterogeneous computing systems.



**Yifan Zhang** received the Ph.D. degree in computer architecture from Huazhong University of Science and Technology, Wuhan, China, in June 2024.

His research interests mainly include field-programmable gate arrays, high-level synthesis, error-correcting codes, storage systems, read voltage calibration in NAND, and the design of compression accelerators.



**Yekang Zhan** is currently pursuing the Ph.D. degree in computer architecture with Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan, China.

His research interests include storage systems, file systems, and computer architecture.



**Jie Yao** (Member, IEEE) received the B.S. degree in computer science and technology, the M.S. degree in computer architecture and the Ph.D. degree in computer architecture from Huazhong University of Science and Technology, Wuhan, China, in 2001, 2004, and 2009, respectively.

He is currently an Associate Professor with Huazhong University of Science and Technology. His research interests include computer architecture, large-scale storage systems, and performance evaluation.

Dr. Yao is a member of the China Computer Federation (CCF) and ACM.